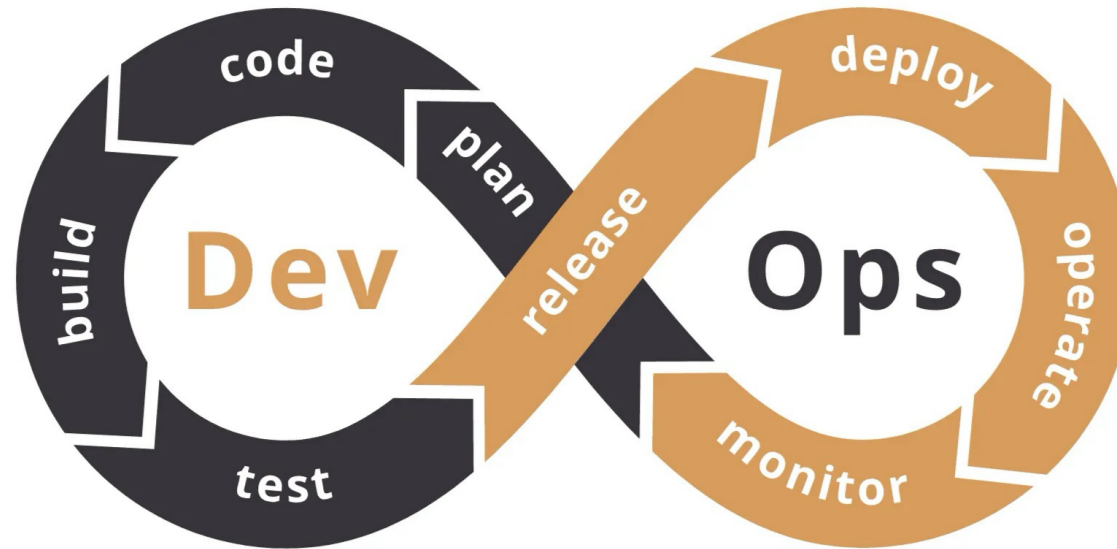




Trainer: Nilesh Ghule

Software Development Methodologies

Module Contents

- ① Software Engg
 - Core concepts
 - SDLC
 - Agile dev model
 - Scrum
 - Atlassian Jira
 - Software Testing
 - Selenium

- ② DevOps
 - SCM: GIT
 - Containerization
 - Docker
 - Orchestration
 - Docker Swarm
 - Kubernetes
 - CI/CD pipeline
 - Jenkins

③ Cloud Computing

DevOps

- ① Development
 - Developers & Testers
 - ↳ Programming, ...
 - App Testing, ...
 - Sw Engg - SDLC
 - Agile - Scrum
- ② Operations
 - Deployments
 - CI/CD mgmt
 - Monitoring

Module Evaluation

- ① CCEE: 25 m
- ② Lab: 20 m
- ③ Internals: 20 m

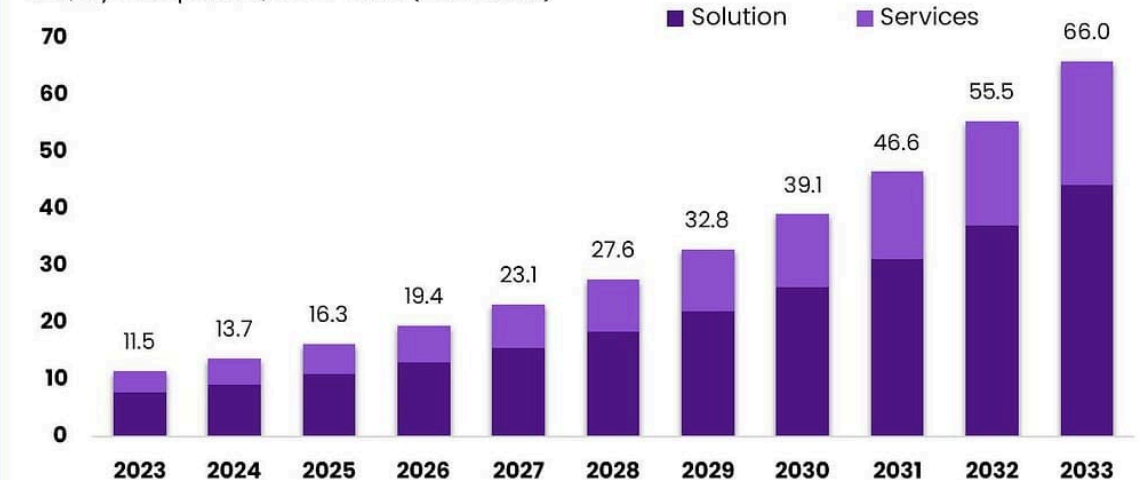
Total: 65 m / 100

Pre requisites

- ① Linux - CLI, basics
- ② Networking - Basics
- ③ Any programming lang/tech
 - Java/JS preferred
- ④ Mindset: Admin, Learning/Exploring, ...

Global DevOps Market

Size, by Component, 2024-2033 (USD Billion)



The Market will Grow
At the CAGR of:

19.1%

The Forecasted Market
Size for 2033 in USD:

\$66.0B

market.us
ONE STOP SHOP FOR THE REPORTS

Software Engineering

Software (a Product)

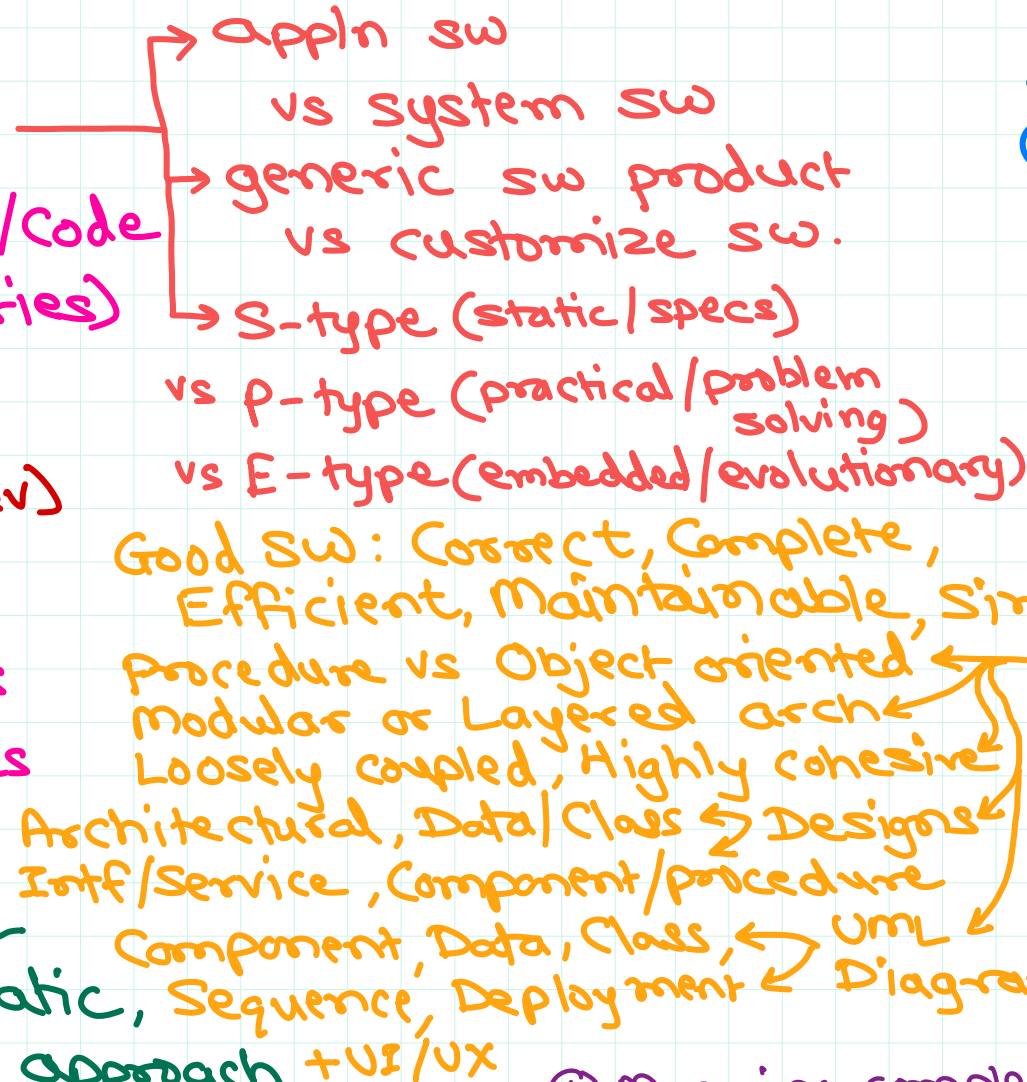
- ① Executable Program/Code
- ② Dependencies (Libraries)
- ③ Documentation

Engineering (Product Dev)

- ① Systematic approach
- ② Well defined principles
- ③ Standard methods/procs

Software Engineering

The appln of a systematic, disciplined, quantifiable approach to the development, operations & maintenance of software i.e. the appln of engineering to the Software (development, ...).



Why SE?

- ① Managing complexity
- ② Cost efficiency
- ③ Reliability & Quality
- ④ Maintainability
- ⑤ Time & Budget compliance
- ⑥ Team coordination

SDLC a.k.a. SDP

① Feasibility Study

② Requirement Gathering

- functional vs non-functional requirement
- [write → analyze]
- [validation ← SRS doc ←]
- Use case diag & Data Flow Diagram

③ Designing

④ Coding / Implementation

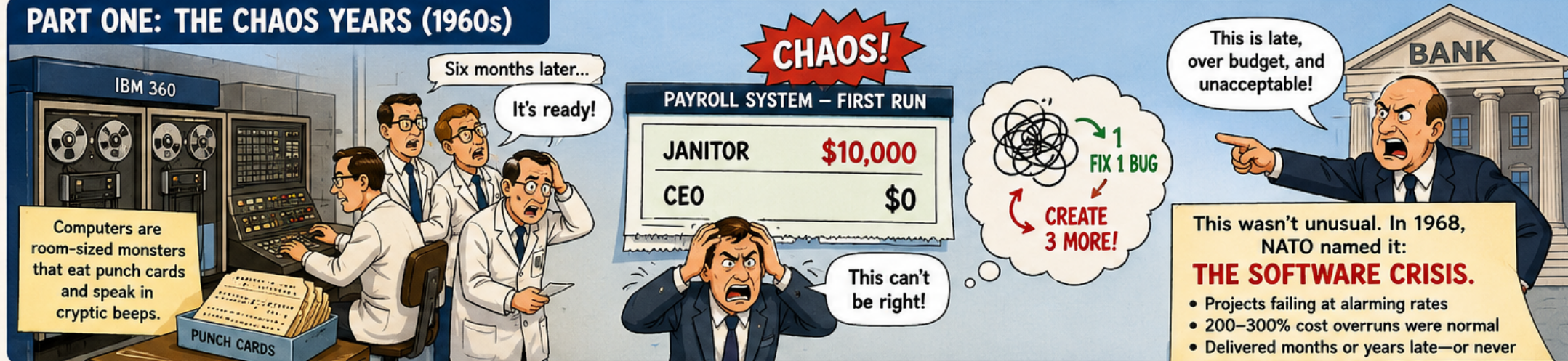
- Principles: SOLID, DRY, KISS, YAGNI.
- Standard coding practice

⑤ Testing

⑥ Deployment / Release

⑦ Maintenance

PART ONE: THE CHAOS YEARS (1960s)



IBM 360

Computers are room-sized monsters that eat punch cards and speak in cryptic beeps.

PUNCH CARDS

Six months later...
It's ready!

CHAOS!

PAYROLL SYSTEM – FIRST RUN

JANITOR	\$10,000
CEO	\$0

This can't be right!

1
FIX 1 BUG
CREATE 3 MORE!

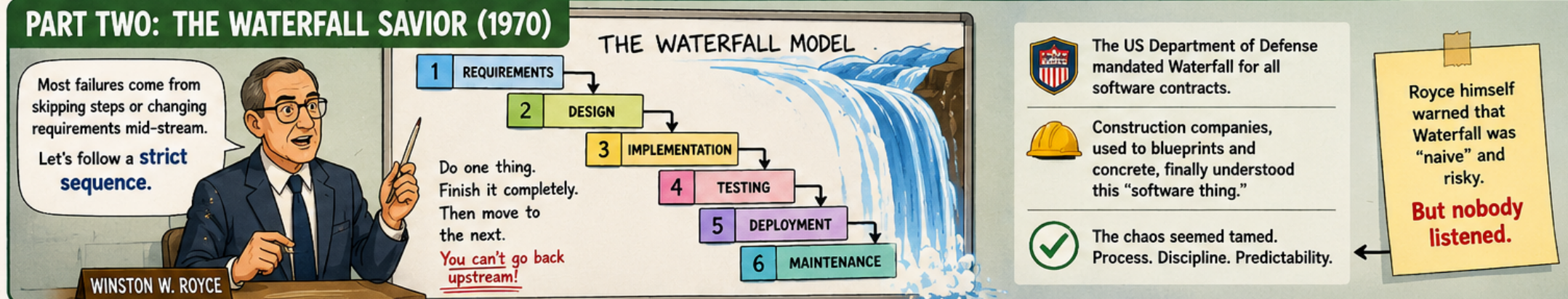
This is late, over budget, and unacceptable!

BANK

This wasn't unusual. In 1968, NATO named it:
THE SOFTWARE CRISIS.

- Projects failing at alarming rates
- 200–300% cost overruns were normal
- Delivered months or years late—or never

PART TWO: THE WATERFALL SAVIOR (1970)



Most failures come from skipping steps or changing requirements mid-stream.
Let's follow a **strict sequence.**

WINSTON W. ROYCE

THE WATERFALL MODEL

- 1 REQUIREMENTS
- 2 DESIGN
- 3 IMPLEMENTATION
- 4 TESTING
- 5 DEPLOYMENT
- 6 MAINTENANCE

Do one thing. Finish it completely. Then move to the next.
You can't go back upstream!

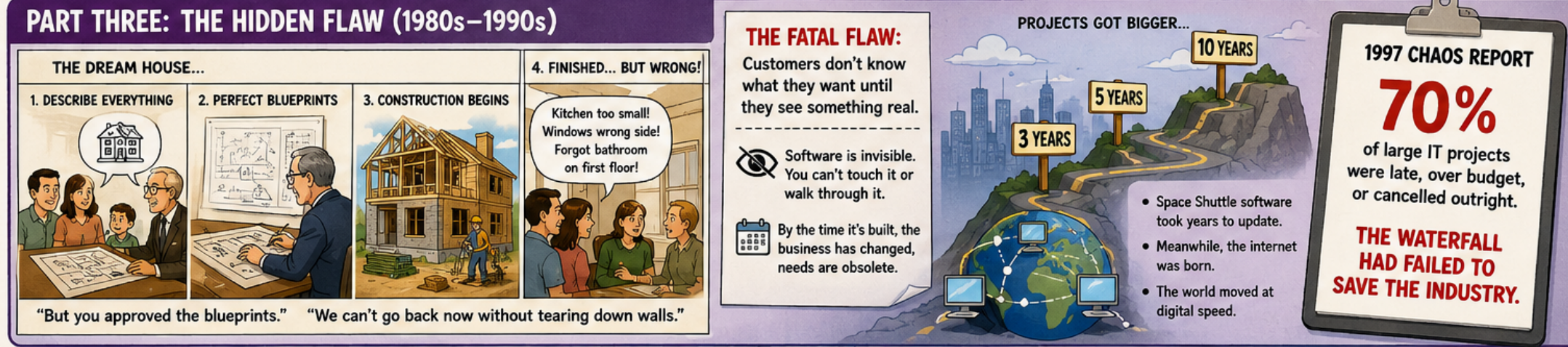
The US Department of Defense mandated Waterfall for all software contracts.

Construction companies, used to blueprints and concrete, finally understood this "software thing."

The chaos seemed tamed. Process. Discipline. Predictability.

Royce himself warned that Waterfall was "naïve" and risky.
But nobody listened.

PART THREE: THE HIDDEN FLAW (1980s–1990s)



THE DREAM HOUSE...

1. DESCRIBE EVERYTHING
2. PERFECT BLUEPRINTS
3. CONSTRUCTION BEGINS
4. FINISHED... BUT WRONG!

"But you approved the blueprints." "We can't go back now without tearing down walls."

Kitchen too small!
Windows wrong side!
Forgot bathroom on first floor!

THE FATAL FLAW:
Customers don't know what they want until they see something real.

Software is invisible. You can't touch it or walk through it.

By the time it's built, the business has changed, needs are obsolete.

PROJECTS GOT BIGGER...

3 YEARS
5 YEARS
10 YEARS

1997 CHAOS REPORT

70%
of large IT projects were late, over budget, or cancelled outright.

THE WATERFALL HAD FAILED TO SAVE THE INDUSTRY.

- Space Shuttle software took years to update.
- Meanwhile, the internet was born.
- The world moved at digital speed.

PART FOUR: THE REBEL ALLIANCE – AGILE IS BORN (2001)



We are uncovering better ways of developing software by doing it and helping others do it.

THE FOUR VALUES

-  Individuals and interactions over processes and tools
-  Working software over comprehensive documentation
-  Customer collaboration over contract negotiation
-  Responding to change over following a plan

-  17 software rebels.  2 days of debate.  Lots of coffee.
-  Different methods. Same frustration: Waterfall was broken.
-  They wrote a short manifesto. It fit on one page.



The name “Agile” came from Mike Beedle’s memory of an article about “agile manufacturing.” It stuck.

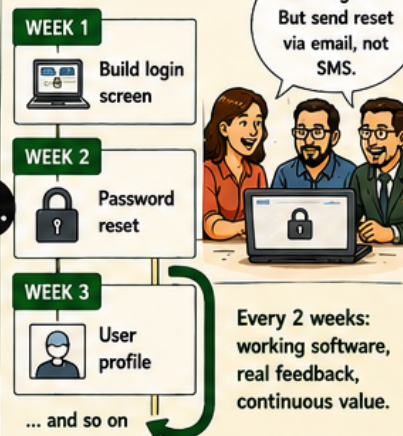
PART FIVE: HOW AGILE WORKS (THE STORY IN PRACTICE)

UNDER WATERFALL



VS.

UNDER AGILE



AGILE IS LIKE COOKING TOGETHER...



SCRUM: Named after a rugby formation.



-  Daily Stand-up (no chairs!)
-  Sprints (2 weeks)
-  Retrospectives (How can we improve?)

KANBAN: From Toyota’s factory floor.

TO DO	DOING	DONE
  	  	  

-  Visual workflow
-  Limit work in progress
-  Focus on flow
-  Deliver continuously

PART SIX: THE AFTERMATH – AGILE WINS, BUT...

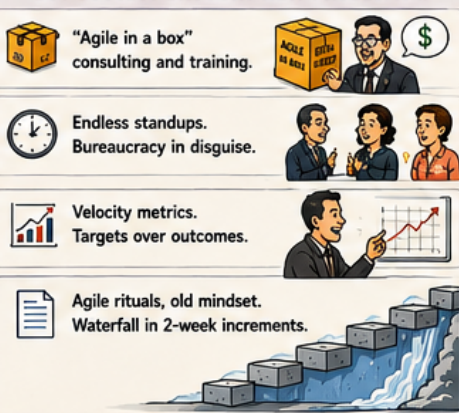
AGILE WINS



-  Faster delivery
-  Happier customers
-  Less waste

 Teams report **60-80% FASTER** time-to-market and higher satisfaction.

BUT SUCCESS BROUGHT NEW PROBLEMS...



THE MANIFESTO SIGNATORIES WARNED:

Mindset, not a rigid process.



“We wanted to stop worshipping plans, not create new rituals.”

WATERFALL ISN'T DEAD

For safety-critical systems (airplane controls, medical devices, etc.), its rigorous up-front analysis is still essential.



AGILE BECAME THE DEFAULT

For web apps, mobile software, and most business systems.



THE RESULTS SPEAK

Faster delivery. Happier customers. Less waste. Better outcomes.



 By 2010, Agile had become the new orthodoxy.



The Waterfall brought discipline to a wild frontier. Agile brought humility. No plan survives first contact with reality. The best way to build something great is to listen, adapt, and deliver something real—early, often, and together.



So, what did you learn? And how can we do better tomorrow?

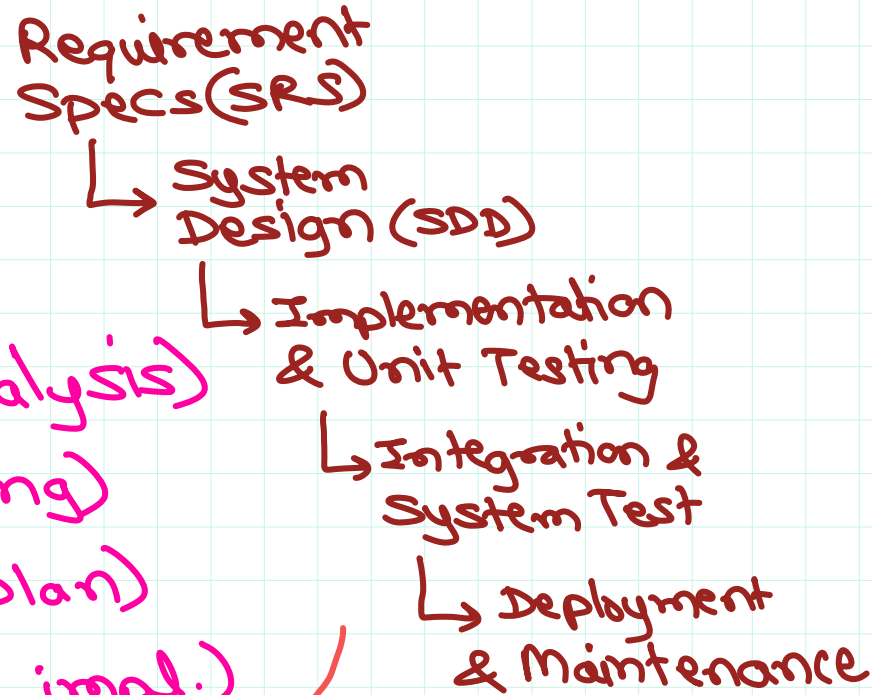


SW Development Models

Dev models

- ① Waterfall
- ② Iterative
- ③ Incremental
- ④ Spiral (Risk analysis)
- ⑤ V-model (Testing)
- ⑥ Big Bang (Zero plan)
- ⑦ Prototyping (mock impl.)
- ⑧ Agile

Waterfall Model



Agile

Think quickly
Reason clearly
Adapt to change

- mindset than methodology
- Agile Manifesto (2001)
 - Individuals & Interactions
x processes & tools
 - Working Software
x comprehensive docs.
 - Customer Collaboration
x contract negotiation
 - Respond to change
x follow the plan

- 12 principles

- ① customer satisfaction
- ② Welcome change (req.)
- ③ Periodic delivery
- ④ Devs + Business People
- ⑤ Value motivated individuals
- ⑥ Face to face conversation
- ⑦ measure = Working Software
- ⑧ sustainable pace
- ⑨ Tech excellence & Good design
- ⑩ max delivery with min efforts
- ⑪ Self organizing teams
- ⑫ Reflect periodically

Agile & Scrum

Agile frameworks: Scrum, Kanban, XP, Scrumban, Lean, ...

Benefits

- ① Customer Satisfaction ++
- ② Faster Time to market
- ③ Higher Quality
- ④ Reduced Risk
- ⑤ Team morale + Transparency +
- ⑥ Predictable Cost & Schedule

Limitations

- ① Lack of Docs → difficult K.T.
- ② Fragmented Design
- ③ Meeting overload
- ④ Customer dependency
- ⑤ Estimate updates with flexibility

Scrum: Framework = Team + Artifacts + Events = Agile impl.

→ Pillars → Transparency + Inspection + Adaption

* Team = Product Owner + Scrum Master + Developers.

* Iterations i.e. Sprints: Fixed length timebox (1-4 weeks)

* User Stories: Features short desc as an end user - by PO.

→ Divided into tasks & subtasks - by devs.

→ may be grouped in Epics.

Events
Scrum → Artifacts

* Product Backlog: ordered evolving collection of stories.

* Sprint Backlog: Stories selected for a sprint.

* Increment: Concrete, usable outcome of a sprint.

* Sprint Planning: User stories picked by Devs as per priority.

* Daily Standup: Yesterday progress + Today's plan + Blockers.

* Review: Demonstrate sprint outcome - end of sprint.

* Retrospective: Sprint Learning - keep up good work + improve on the mistakes

* CI-CD: recommended for incremental work.

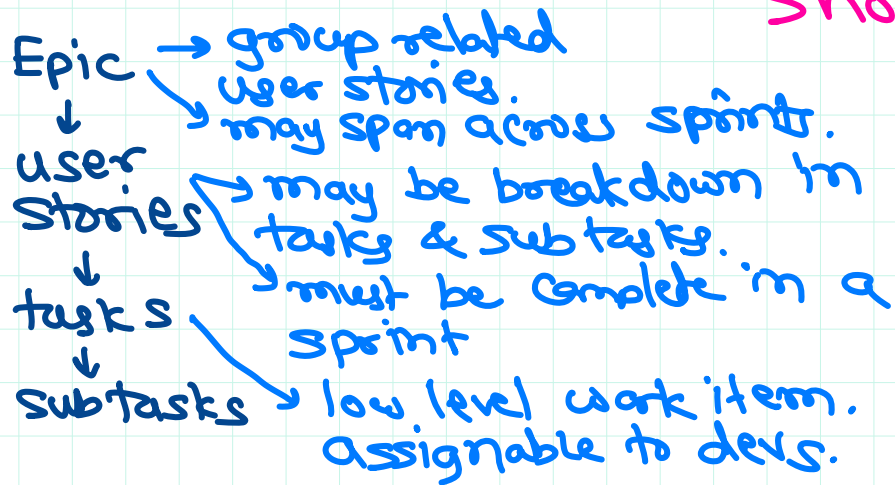
Scrum - Agile Framework



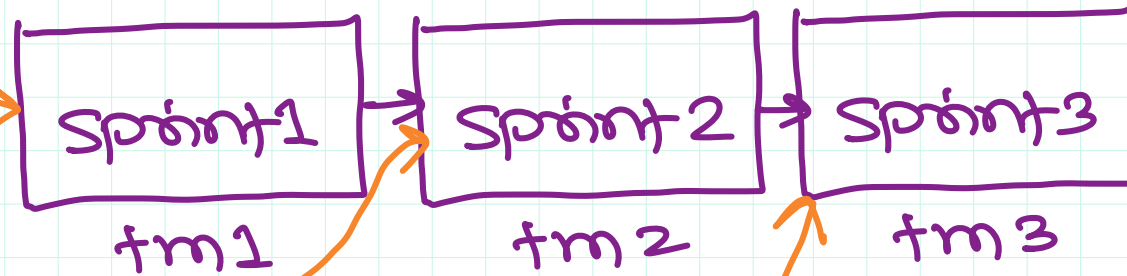
Artifact

- ① Product backlog
- ② Sprint backlog
- ③ Increment

work item
(issue)



Sprint ♥



goal, pick req. → design
↓
show ← test ← impl, outcome

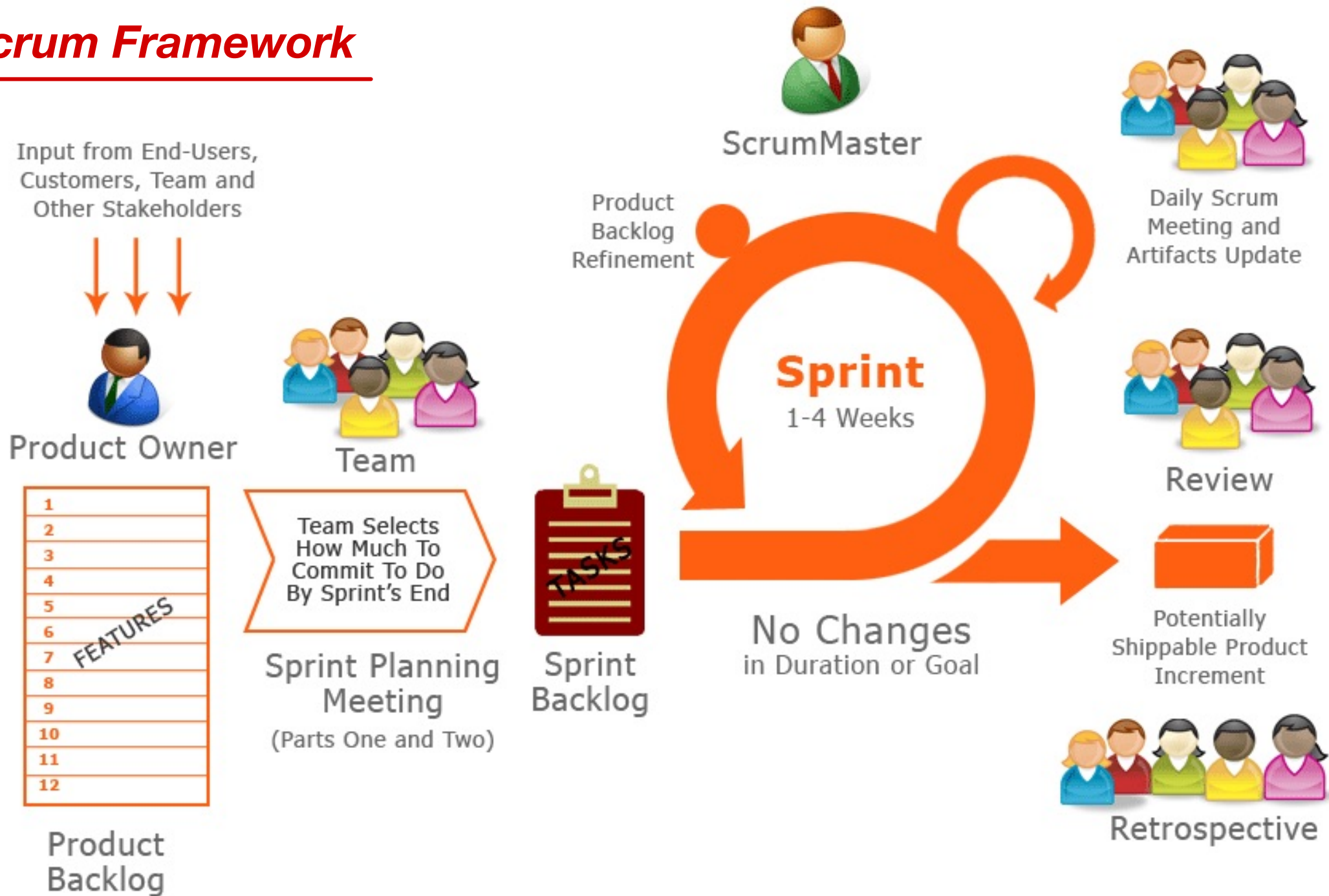
Events

- sprint planning
- Daily standups
- sprint review
- sprint retrospective

Team

- ① product owner
- ② Developers & Testers
- ③ Scrum Master

Scrum Framework



WATERFALL MODEL FOR AN E-COMMERCE APPLICATION



EXAMPLE: E-COMMERCE APP

- User login ✓
- Product catalog ✓
- Cart ✓
- Payments ✓
- Orders ✓
- Admin dashboard ✓



- Login should support email/password
- Product page should show images
- Cart should allow quantity updates
- Payment via Razorpay/Stripe
- Admin can manage inventory

Everything is finalized upfront.

DATABASE SCHEMA



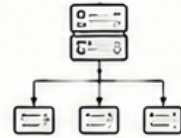
APIs



UI WIREFRAMES



SYSTEM ARCHITECTURE



Everything is planned before coding.

BACKEND DEVELOPMENT



- User APIs
- Cart APIs
- Payment APIs
- Order APIs
- Admin APIs



FRONTEND DEVELOPMENT



- React UI
- Checkout pages
- Admin screens
- Product pages

Developers code the full application. No customer feedback yet.

- ✓ Functional Testing
- ✓ UI/UX Testing
- ✓ Integration Testing
- ✓ Performance Testing
- ✓ Security Testing



QA team tests everything after development finishes. Usually many bugs appear late.



Entire project goes live together.

PROBLEMS IN WATERFALL

1 LATE FEEDBACK



Customer sees product after many months.

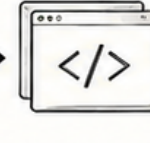


Hmm... We don't like the UX.

2 REQUIREMENT CHANGES ARE PAINFUL



We now want wishlist feature!



Modern businesses change frequently. Changes after design and development are completed become expensive and time-consuming.

3 HIGH RISK



If architecture is wrong, you discover very late.



Why is the system so slow and unstable?

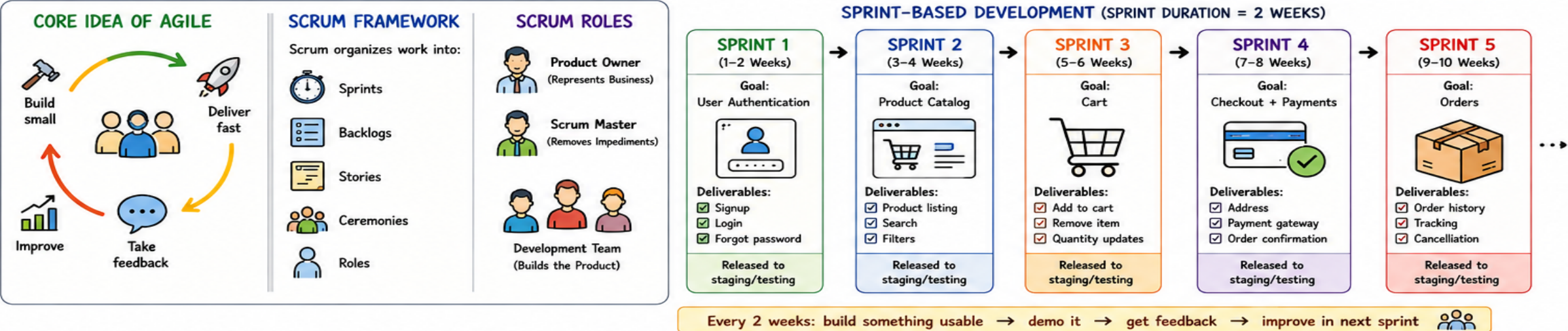


Issues in architecture, performance, or scalability are discovered very late in the project.

AGILE SCRUM MODEL FOR AN E-COMMERCE APPLICATION

**EXAMPLE: E-COMMERCE APP**

- User login
- Product catalog
- Cart
- Payments
- Orders
- Admin dashboard



EXAMPLE USER STORIES

LOGIN STORY

As a customer
I want to login using email/password
So that I can access my orders.

SEARCH STORY

As a customer
I want to search products
So that I can quickly find items.

ACCEPTANCE CRITERIA

Conditions that define completion.
Example for Login Story:

- ✓ User can login with valid credentials
- ✓ Error shown for invalid password
- ✓ JWT token generated
- ✓ Session persists after refresh

QA uses acceptance criteria to verify the story.

TASKS

Technical implementation work.
One story is divided into tasks.

Story: User Login

- ✓ Design login UI (UI Dev)
- ✓ Create login API (Backend Dev)
- ✓ Implement JWT authentication (Backend Dev)
- ✓ Add validation (Backend Dev)
- ✓ Write unit tests (Dev)
- ✓ QA testing (QA Engineer)

Tasks are estimated and assigned to team members.

SPRINT

Fixed development cycle.
Usually 1-2 weeks.

Example Sprint

SPRINT 3
1 May → 14 May
Goal: Cart Functionality

Stories in this sprint:

- ✓ Add to cart
- ✓ Remove from cart
- ✓ Update quantity

REAL PRACTICAL FLOW (END-TO-END)

EXAMPLE: "ADD WISHLIST FEATURE"

BUSINESS REQUIREMENT

PO says:
Customers should save products for later purchase.

EPIC CREATION

Epic:
Wishlist System

USER STORIES

Story 1:
Add item to wishlist
Story 2:
Remove item from wishlist
Story 3:
View wishlist page

SPRINT PLANNING

Sprint 7 includes:
• Add wishlist API
• Wishlist UI
• Database table

TASKS

Backend:
• Create wishlist table
• Build REST APIs
Frontend:
• Add heart icon
• Wishlist page UI
• Functional testing

DEVELOPMENT (DAILY WORK)

Team works daily.
Standups happen.

REVIEW

Demo to stakeholders.
Feedback: Need wishlist sharing.
New story added to backlog.

SCRUM CEREMONIES (HAPPEN EVERY SPRINT)

SPRINT PLANNING

Team selects stories from backlog and plans how to deliver.

DAILY STANDUP

15-minute daily sync.
What I did, What I will do, Impediments.

SPRINT REVIEW

Team demos completed work to stakeholders and gets feedback.

SPRINT RETROSPECTIVE

Team reflects on what went well, what didn't, and how to improve.

BENEFITS OF SCRUM FOR E-COMMERCE

- Early & frequent value delivery
- Adapt quickly to changes
- Higher customer satisfaction
- Lower risk
- Better visibility and control

Atlassian Jira → Agile Project Mgmt Tool



↳ Scrum, Kanban, Scaled & Mixed.

Jira Structure

Organization / Site

↓ users, billing, security, settings, ...
Space / Project → appn:

↓ backlog, sprints, reports, ...
Board → Visual workflow:

↓ Scrum/Kanban board
↓ Lanes: To Do / In Progress / Done.

Work Items / Issues

↳ Epic: Business Feature/module
across 1/more sprints.

Set of stories/tasks

↳ Story / Task / Bug:

User story or internal work or defect
estimable & prioritized
completed in a sprint

↳ SubTask: Doable work unit (check list) in task or story or bug.

Jira highlights for beginners

Backlog
Sprint
Board
Ticket/Issue
Reports

Product backlog

- set of stories/tasks
- defined by P.O + prioritized

Sprint Planning

- Sprint backlog by Devs
- Sprint goal
- Assign work to Devs who will update status in active sprint

Reports

- Sprint Report
- Burndown chart (planned & completed work)
- Velocity chart (work finish per sprint)

Dashboard

- My: open issues & assigned tasks
- Sprint Burndown & recent updates

Automation



Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>